Testat Spickzettel – Versuch 1, Teil 1 (Variablen.c)

Almanca cümleleri ezberle, Türkçe kısımlar sadece anlaman için. Kaynak: Skript Kap. 1.1 ve 2.1.4.

0. Gesamtzusammenfassung (30 Sekunden, wenn sie sagen: „Erklären Sie Ihr Programm“)

Auswendig lernen:
„Mein Programm **Variablen.c** liest zwei ganze Zahlen ein und berechnet Summe und Quotient: zuerst mit **int**, dann mit **double**, dann mit **explizitem Cast**. So zeige ich, dass die Division mit zwei int eine **Ganzzahldivision** ist und die Nachkommastellen verloren gehen.
Danach lese ich Vor- und Nachnamen ein und setze sie mit **strcpy** und **strcat** in zwei Formen zusammen. In einem eigenen **Block** zeige ich ein ein- und ein zweidimensionales **Feld** und eine **Konstante**, um den **Gültigkeitsbereich** zu zeigen.
Zum Schluss wandle ich die ersten beiden Buchstaben des Vornamens in **ASCII-Zahlen** um und berechne mit **modulo 32** ihre Position im Alphabet, unabhängig von Groß-/Kleinschreibung.“

Türkçe özet: İki sayı al, int / double / cast ile topla-böl (farkı göster). Ad-soyadı strcpy/strcat ile birleştir. Blokta dizi ve const göster. Harfleri ASCII'ye çevirip % 32 ile alfabe sırasını bul.

1. Die 7 Merksätze (bunlar yeter!)

#	Thema	Merksatz (Almanca – aynen söyle)	Türkçe
1	int / int	„Beide Operanden sind int, also ist die Division eine **Ganzzahldivision**. Die Nachkommastellen gehen verloren.“	7/2 = 3
2	double = int/int	„Die Division passiert **zuerst** mit int. Erst danach wird das Ergebnis **implizit** in double umgewandelt.“	Hâlâ 3.000000
3	Cast	„Mit dem Cast werden die Operanden **vor** der Division zu double. Deshalb ergibt 7 / 2 dann 3.5.“	3.5
4	String	„Ein String in C ist ein **char-Feld**, das mit dem Nullzeichen **'\0'** endet.“	char[50] = 49 harf + \0
5	scanf &	„Das **&** ist der Adressoperator. scanf muss wissen, **wo im Speicher** der Wert abgelegt wird.“	Değişkenin adresi
6	Block / const	„Ein Name ist nur in dem **Block** gültig, in dem er definiert wurde. Es gilt die **innerste** Definition.“	İç blok dış değişkeni gizler
7	% 32	„ASCII: 'A' ist 65, 'a' ist 97. Beide ergeben **modulo 32** den Wert 1. Das ist unabhängig von Groß-/Kleinschreibung.“	65%32 = 97%32 = 1

2. Code in einem Blick (Aufgabe 1–7)

```
1  int iSumme    = iErste + iZweite;       // int
   int iQuotient = iErste / iZweite;       // 7/2 = 3
2  double dSumme    = iErste + iZweite;
   double dQuotient = iErste / iZweite;    // 3.000000 !
3  double dQuotientCast =
       (double) iErste / (double) iZweite; // 3.5
4  strcpy(vornameName, vorname);           // kopieren
   strcat(vornameName, " ");               // anhängen
   strcat(vornameName, nachname);          // "Vorname Nachname"
   // nameVorname: erst nachname, dann ", ", dann vorname
5a int iFeld[2] = { 1, 2 };
5b int spielfeld[2][3] = {{1,2,3},{4,5,6}};
   for y 0..1 { for x 0..2 { printf("%d ", spielfeld[y][x]); }
                printf("\n"); }
5c { const int iZweite = 1; ... }          // im Block = 1
   printf(iZweite);                        // danach = Eingabewert
6  int iName1 = vorname[0];                // ASCII-Zahl
7  int iPosition1 = iName1 % 32;           // Position im Alphabet
```

2b. Was macht der Code? (Schritt für Schritt erklären)

Nr.	Was passiert?	So sagst du es (Deutsch)
1	Zwei ints einlesen, Summe und Quotient als int	„Ich lese iErste und iZweite mit scanf ein und speichere Summe und Quotient in int-Variablen. Der Quotient ist abgeschnitten.“
2	Gleiche Rechnung, Ergebnis in double gespeichert	„Das Ergebnis wird erst als int berechnet und dann implizit nach double umgewandelt. Die Nachkommastellen sind trotzdem weg.“
3	Operanden vorher mit (double) umwandeln	„Durch den expliziten Cast rechnet der Computer mit double. Der Quotient hat jetzt Nachkommastellen.“
4	Vor-/Nachname einlesen, zwei Namensformen bauen	„Ich kopiere mit strcpy den Vornamen, hänge mit strcat ein Leerzeichen und den Nachnamen an. Für 'Name, Vorname' beginne ich mit dem Nachnamen und hänge ', ' und den Vornamen an.“
5a	Feld iFeld mit 1 und 2 ausgeben	„Ein eindimensionales Feld mit zwei Elementen, Index 0 und 1.“
5b	2x3-Feld mit zwei for-Schleifen ausgeben	„Die äußere Schleife läuft über die Zeilen, die innere über die Spalten. Nach jeder Zeile gibt es einen Zeilenumbruch.“
5c	const iZweite = 1 im Block, danach außen nochmal	„Im Block verdeckt die Konstante die äußere Variable. Nach dem Block ist wieder die äußere sichtbar, deshalb steht dort der eingegebene Wert.“
6	vorname[0] und vorname[1] in int speichern	„Ein char ist intern eine Zahl. Die Zuweisung an ein int liefert den ASCII-Code.“
7	iName % 32	„Groß- und Kleinbuchstaben unterscheiden sich im ASCII-Code um 32. Modulo 32 liefert für beide dieselbe Position, A und a ergeben 1.“

Hinweis zu meinem Code (falls gefragt):
Ist iZweite = 0, würde die Division durch null das Programm abstürzen lassen. Das fange ich in meinem Code nicht ab. Die Zielfelder vornameName (101) und nameVorname (102) sind größer als vorname/nachname (je 50), damit strcat nicht über das Feld hinausschreibt.

3. Die Lieblingsfrage: Warum unterscheiden sich die Ergebnisse?

Antwort in 3 Sätzen (auswendig lernen):
1) Ohne Cast sind beide Operanden **int**, die Division ist eine **Ganzzahldivision**.
2) Das Ergebnis ist schon abgeschnitten, bevor es nach double umgewandelt wird.
3) Mit Cast werden die Operanden **vor** der Division zu double, also bleiben die Nachkommastellen erhalten.
Beispiel: 7 und 2: ohne Cast 3.000000, mit Cast 3.500000. Bei der Summe sieht man keinen Unterschied.

Zusatz (oft gefragt):
„Umwandlung bedeutet **nicht**, dass sich der Datentyp der Variable ändert. iErste bleibt ein int, nur der gelieferte **Wert** ist ein double.“
implizit = automatisch durch den Compiler. **explizit** = vom Programmierer gewollt mit (double). Beim expliziten Cast gibt es **keine Warnung**.

4. Gültigkeitsbereich bei 5c (häufige Frage)

```
int iZweite = <Eingabe>;      // außen
{
    const int iZweite = 1;    // NEUE Variable, verdeckt die äußere
    printf(iZweite);          // -> 1
```

```
}
printf(iZweite);              // -> Eingabewert (äußere wieder sichtbar)
```

Merksatz:
„Im Block ist es eine **andere** Variable mit gleichem Namen. Die äußere behält ihren Wert, ist im Block aber nicht sichtbar. Nach dem Block ist wieder die äußere gültig.“
const = nicht mehr änderbar, muss **sofort initialisiert** werden.

5. Strings: Was musst du erklären können?

Frage	Antwort
Warum kein & bei scanf("%49s", vorname)?	Ein Feld zerfällt zu einem **Zeiger auf sein erstes Element** (Array-Decay). Der Feldname ist schon die Adresse.
Warum %49s und nicht %s?	Begrenzt die Eingabe: 49 Zeichen + '\0' passen in char[50].
Warum strcpy statt =?	Feste char-Felder kann man nicht mit = zuweisen, nur zeichenweise oder mit strcpy.
Warum getrennt Vor- und Nachname?	scanf mit %s hört beim **ersten Leerzeichen** auf.
Gefahr bei strcpy/strcat?	Sie prüfen **nicht**, ob das Zielfeld groß genug ist. Zu lange Texte überschreiben den Speicher dahinter. Deshalb sind meine Zielfelder größer (101/102).
strlen?	Länge der Zeichenkette **ohne** Nullzeichen.

6. Platzhalter und Datentypen

%d	%f	%lf	%c	%s	\n
int	float/double (printf)	double (scanf)	char	String	Zeilenumbruch

char	short	int	long long	float	double	bool
1 Byte	2 Byte	4 Byte	8 Byte	4 Byte	8 Byte	1 Byte (stdbool.h)

Falscher Platzhalter (z. B. %d bei double): Compiler meldet nicht zuverlässig einen Fehler, die Ausgabe ist sinnlos.

7. Kurzfragen (Frage -> Antwort)

Frage	Antwort
Was ist main?	Einstiegspunkt, genau eine pro Programm.
Was macht return 0?	Rückgabe an das Betriebssystem, 0 = fehlerfrei.
Wozu #include <stdio.h>?	Deklarationen von printf/scanf. Die Implementierung kommt beim **Linken**.
Deklaration vs. Definition?	Definition reserviert Speicher (int x;). Deklaration sagt nur, wie ein Typ/eine Funktion aufgebaut ist.
Array-Index?	Beginnt bei **0**, endet bei n-1. Größe fest = statische Datenstruktur.
2D-Zugriff?	feld[y][x], **nicht** feld[y,x].
Was ist %?	Modulo, Divisionsrest (5%3=2), nur für ganzzahlige Typen.
= vs. ==?	= Zuweisung, == Vergleich. Häufige Fehlerquelle.
i++ vs. ++i?	Postfix: erst Wert benutzen, dann erhöhen. Präfix: erst erhöhen, dann benutzen.
Was ist ein Block?	Anweisungen in { }, syntaktisch wie eine einzelne Anweisung.

Frage	Antwort
int = double?	Implizit mit Informationsverlust: Compiler-Warnung, Nachkommastellen weg.
Warum initialisieren?	Um Fehler durch uninitialisierte Variablen zu vermeiden (Zufallswert).

8. Code Convention (wird mitbewertet)

- **lowerCamelCase** (iErste, vornameName), aussagekräftige Namen
- Variablen bei der Definition **initialisieren**
- { } stehen **allein in einer Zeile**, Code eingerückt
- Ausreichend kommentieren, mehrzeilig mit /* und */
- Was sich nicht ändert: **const**

9. Ablauf Testat

Kurze mündliche Prüfung auf Deutsch, **ohne Computer**. Kleine Live-Programmieraufgabe + Fragen zum Skript + Fragen zu deinem Code. Volle Punkte nur, wenn du den Code **selbstständig erklären** kannst. Üb auf Papier: Quotient mit Cast, strcpy/strcat, 2D-Feld mit zwei for-Schleifen, % 32.